MicroBlend Database Overview

27 tables across 7 domain areas. Every table includes `created_at` and `updated_at` (from a shared `BaseModel`).

Domains

1. Users & Notifications

- **users** — Central actor table. Supports guests (is_guest, guest_expires_at) and staff (role, staff_role). Soft-delete via is_deleted.
- **notifications** — Targeted at a user (recipient_id FK) or a role (role_target string). Category for client-side filtering.
- **debounce_records** — Idempotency guard. Unique on (actor_key, action, object_key). No FKs — intentionally loose for speed.

2. Menu & Ingredients

- **categories** → **menu_items** (1:N). Category can be null (SET_NULL on delete).
- **menu_items** ↔ **ingredients** via **menu_item_ingredients** (M:N with payload). Junction has quantity_required and a unique pair constraint.
- **ingredients** — Tracks unit (g/ml/pc) and reorder_level for low-stock alerts.

3. Inventory

- **ingredients** → **inventory_batches** (1:N). Each batch records quantity_added, quantity_remaining, unit_cost, and expiration_date.
- **ingredients** → **inventory_movements** (1:N) — every restock/deduct/adjust is logged.
- **inventory_batches** → **inventory_movements** (1:N, optional) — a movement can optionally point to a specific batch.
- **inventory_movements** also FKs to **users** (actor) and has a denormalized related_order_id (not a FK, just an ID).

4. Orders

- **orders** — Central to the system. Links to table_session and placed_by (both nullable SET_NULL). Has three parallel status tracks: kitchen_status, bar_status, cashier_status. Receipt numbers are auto-generated unique.
- **order_items** — Line items. Denormalizes item_name and station from the menu item at time of order. FK to menu_items is nullable (SET_NULL) so order history survives menu changes.
- **order_status_logs** — Immutable audit trail for status changes. Stores actor, status, note, and a metadata JSON blob.

- **feedback_entries** — FK to both users (submitter) and orders. Supports two entry types: feedback and report.

5. Order Playlists

- **order_playlists** — A named saved list owned by a user. Unique on (owner_id, name).
- **order_playlist_items** — Junction between playlist and menu_item with quantity. Unique on (playlist_id, menu_item_id).

6. Table Management

- **tables** — Physical tables with identifier (display name), capacity, status (vacant/occupied/merged/blocked), zone, and a unique auto-generated QR code value.
- **table_merge_groups** — Groups tables together (M:N via **table_merge_group_tables** junction).
- **table_sessions** — A seating session. Links to table, merge_group, two user FKs (opened_by, customer_account), and an optional scan_request. Party_size and guest_label for guest tracking. is_active + started_at/ended_at for lifecycle.
- **table_scan_requests** — QR scan intent. Has status (pending/approved/blocked) and tracks the requesting device.
- **staff_page_requests** — Customer calls staff. Links to table and session, with reason (cleanup/payment/accident) and status (pending/finished).

7. Analytics, Integration & System

- **audit_logs** — Generic action log. Stores actor, action, and polymorphic target (target_type + target_id + target_label).
- **generated_reports** — Report generation requests with range_type (daily/weekly/etc.), date range, and JSON payload.
- **cost_simulations** — What-if scenarios storing assumptions and results as JSON.
- **external_systems → sync_events** (1:N). Sync_events is an outbox: each event has an event_type, aggregate identifiers, and an idempotency_key for at-least-once delivery.
- **realtime_events** — WebSocket event log. User FK is nullable so system events can fire without a specific user.

Key Design Patterns

Pattern	**Where**
Soft delete	users (is_deleted)
Polymorphic FK	audit_logs (target_type + target_id), inventory_movements (related_order_id)
Parallel status tracks	orders (kitchen/bar/cashier status)
Denormalized copy	order_items.item_name (survives menu

Pattern	Where
	changes)
Idempotency guard	debounce_records, sync_events.idempotency_key
Outbox pattern	sync_events
JSON payload for flexibility	audit_logs, generated_reports, cost_simulations, sync_events, realtime_events, order_status_logs
M:N with payload	menu_item_ingredients (quantity_required), order_playlist_items (quantity)
Implicit M:N	table_merge_groups ↔ tables (junction table auto-generated by Django)